# 1 Binary Search Trees:

**NOTE:**

If the number of nodes in the tree is $n$ then the number of binary tree constructed from the given number of node $n$ is:

$$2^n - n$$

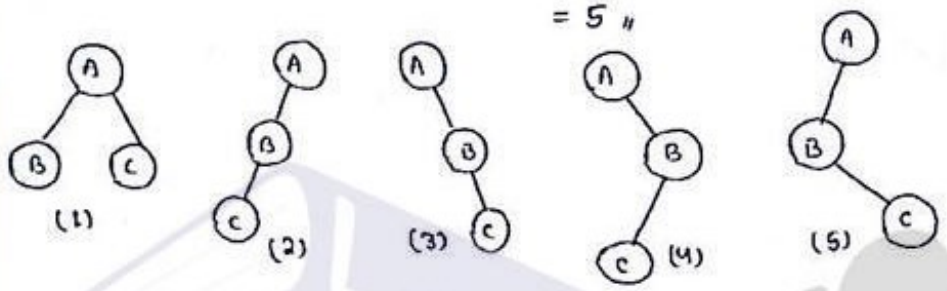
**Example:** no. of nodes = 3 = n

no. of possible binary tree $= 2^n - n$

$= 2^3 - 3$

$= 8 - 3$

$= 5$ //

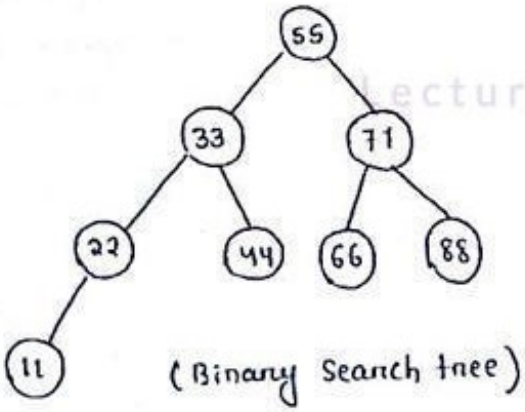



## Binary Search Tree: (BST)

**Definition:** A binary tree T is termed as binary search tree (BST) or binary sorted tree if each node n of T satisfies the following property.

1) The value at n is greater than the values of all nodes in its left subtree and
2) The value at n is less than the values of all nodes in its right subtree.

**Example:**




(Binary Search tree)

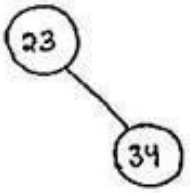
## Creation of BST from the given nodes :

Nodes are : 23 34 35 100 49 53 36 L

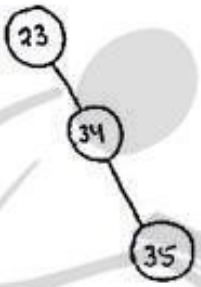
Step1 : if root = NULL then
root = 23


Step2 : if (34 > 23) and
if root → right = NULL then
root → right = 34

else if root → left = NULL then
root → left = 34 .

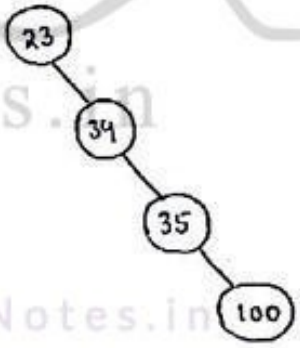



Step3 : 35 > 23
if root → right = = '10'
No.
if 35 > 34 and
if 34 → right = NULL
34 → right = 35




Step4 : 100 > 23
root → right = '10'. (No)
100 > 34
root → right = NULL (No)

100 > 35
35 → right = NULL
∴ 35 → right = 100



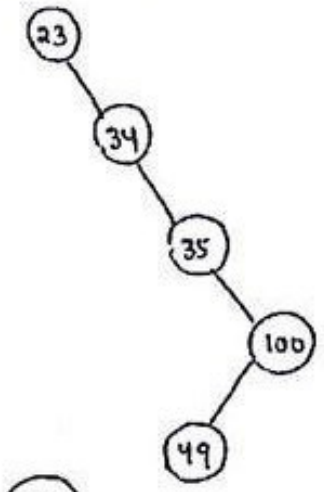

Step5 : 49 > 100 23
root → right = NULL (No)

49 > 34
34 → right = NULL (No)

49 > 35
35 → right = NULL (No)

49 > 100 (No)

100 → left = 49




Step6 : 53 > 23

23 → right ≠ NULL

53 > 34

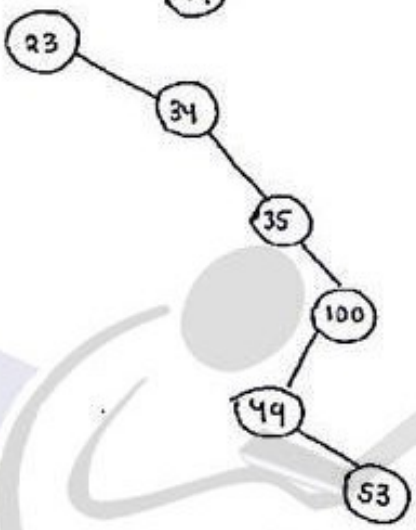
34 → right ≠ NULL

53 > 35 , 35 → right ≠ NULL

53 > 100 (No)

100 → left ≠ NULL

53 > 49 , 49 → right = 53




Step7 : 36 > 23

23 → right ≠ null

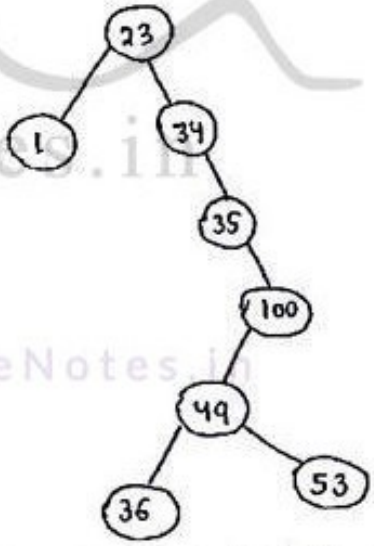
36 > 34 , 34 → right ≠ null

36 > 35 , 35 → right ≠ null

36 > 100 (No)

100 → left ≠ null

36 > 49 (No), 49 → left = 36




Step8 : 1 > 23 (No)

23 → left = NULL , So

23 → left = 1

NOTE : If we allow the repeatation of data elements in BST , then it should satisfy the following condition.

i) Each node n > left(n)

ii) Each node n <= Right(n).

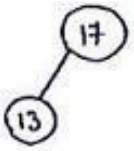
Construct the Binary Search Tree from the following sequence of integers.

17, 13, 14, 18, 30, 6, 5, 35, 12, 8

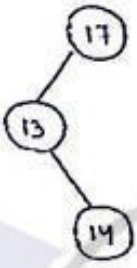
Soln: Integers are : 17, 13, 14, 18, 30, 6, 5, 35, 12, 8

1) root = 17

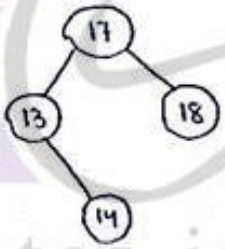

2) 13 > 17 (No)
17 → left = 13



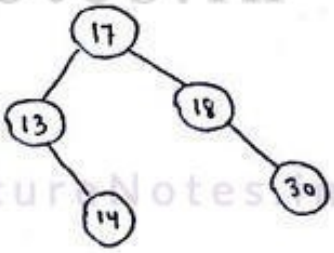

3) 14 > 17 (No)
17 → left ≠ Null
17 → left = 13
so 14 > 13
13 → right = 14



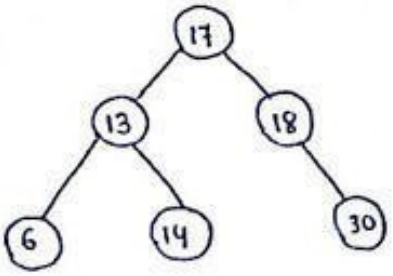

4) 18 > 17 (yes)
17 → right = NULL so
17 → right = 18




5) 30 > 17 (yes)
17 → right ≠ null (18)
30 > 18 (yes)
18 → right = 30




6) 6 > 17 (No).
17 → left ≠ Null (13)
6 > 13 (No)
13 → left = 6



7) 5 > 17 (No)

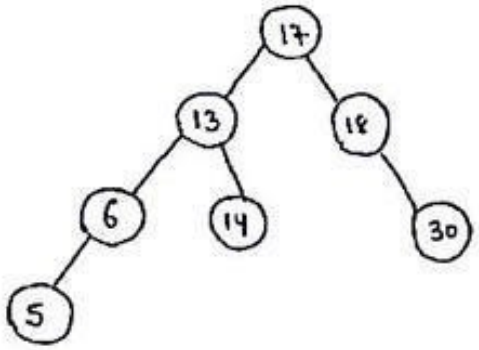
17 → left ≠ null (13)

5 > 13 (No)

13 → left ≠ null (6)

5 > 6 (No)

6 → left = 5




8) 35 > 17 (Yes)

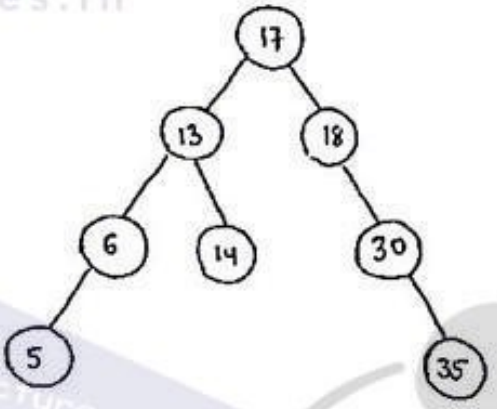
17 → right ≠ null (18)

35 > 18 (Yes)

18 → right ≠ null (30)

35 > 30 (Yes)

30 → right = 35




9) 12 > 17 (No)

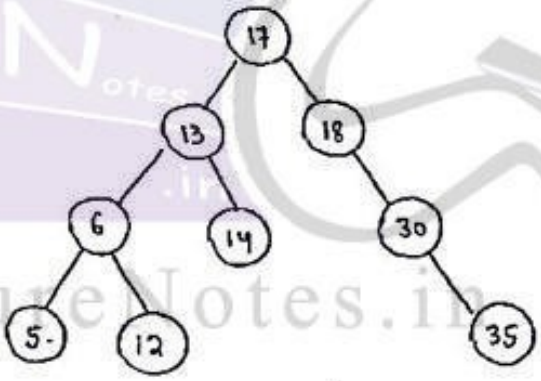
17 → left ≠ null (13)

12 > 13 (No)

13 → left ≠ null (6)

12 > 6 (Yes)

6 → right = 12




10) 8 > 17 (No)

17 → left ≠ null (13)

8 > 13 (No)

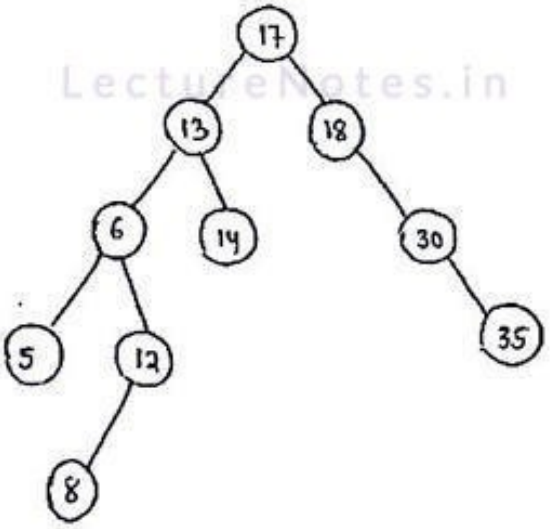
13 → left ≠ null (6)

8 > 6 (Yes)

6 → right ≠ null (12)

8 > 12 (No)

12 → left = 8



## Operations In BST :

Operations in Binary search tree includes

i) Insertion
ii) Deletion.
iii) Searching.

### Insertion :

Algorithm :

PROCEDURE Insertion-in-BST (root, nw, info, R-child, L-child, ptr)

// root : pointer variable which holds the address of 1st node in BST.
// nw : pointer variable which holds the address of newly created node.
// info : variable for holding the information part of each node.
// R-child : pointer variable which holds the address of right child of a given node.
// L-child : pointer variable which holds the address of left child of a given node.
// ptr : temporary pointer variable.

Step1 : if (root = null) then
  i) root = nw.
  ii) goto step 5

Step2 : else
  ptr = root.

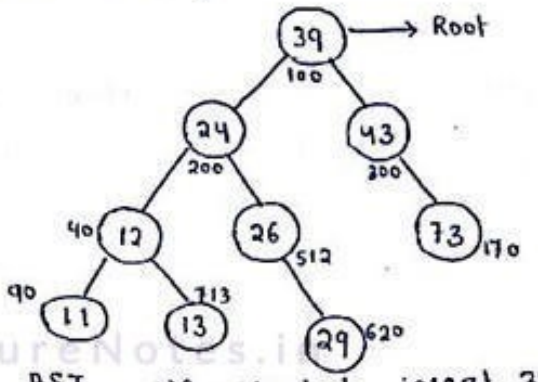
Step3 : if (ptr → info > nw → info) then
  i) move to left child of ptr and check if (ptr → left = null) then set ptr → left = nw.
  ii) else move to begining of step 3.

Step4 : If (ptr → info < nw → info) then
  i) move to right child of ptr and check if (ptr → right = null) then set ptr → right = nw.
  ii) else move to begining of step3.

Step5 : return root.

Step 6 : Exit

Example: Given a BST

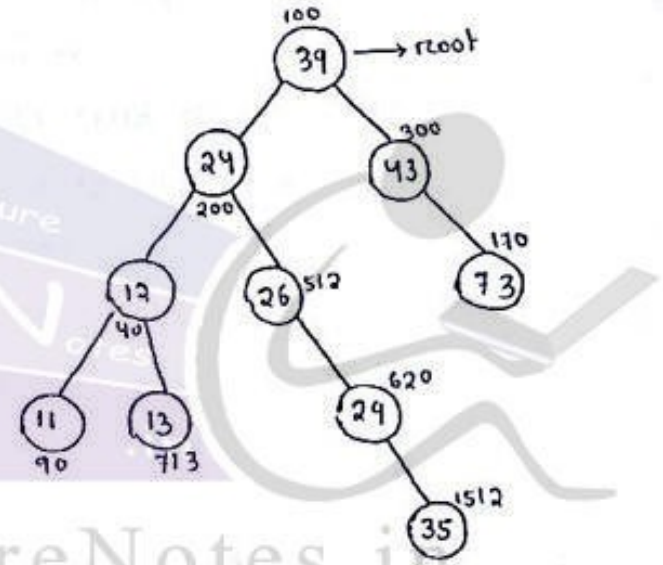



nw


In the above BST, we want to insert 35

1) Root = null (No)

So ptr = 100




2) if ( 39 > 35 )

ptr = 200.

if ( 200 = null ) (No).

if ( 24 > 35 ) (No)

if ( 24 < 35 ) then

ptr = 512

if ( 512 = null ) (no)

if ( 26 < 35 ) (yes)

ptr = 620

if ( 620 = null ) (no)

if ( 29 < 35 ) then

ptr = null

if ( ptr → right = 35

## Deletion :

### Algorithm :

PROCEDURE Delete-node-in-BST ( N, L(N), R(N), P(N))

// N : it is the node to be deleted.

// L(N) : left child of node N.

// R(N) : Right child of node N.

// P(N) : parent node of N.

Step1 : If the node 'N' is not having any child node then.

a) If 'N' is the right child of P(N) then

i) Assign null to right part of P(N).

ii) delete node N.

b) if 'N' is the Left child of P(N) then

i) Assign null to left part of P(N)

ii) delete node N.

Step2 : If there exist only one child node of N then

a) If 'N' is the right child R(N) then

P(N) → right = child node of N.

b) else

P(N) → left = child node of N.

c) delete (N).

Step3 : If node 'N' having 2 child node then

a) goto the right subtree of N and findout the smallest value in right subtree.

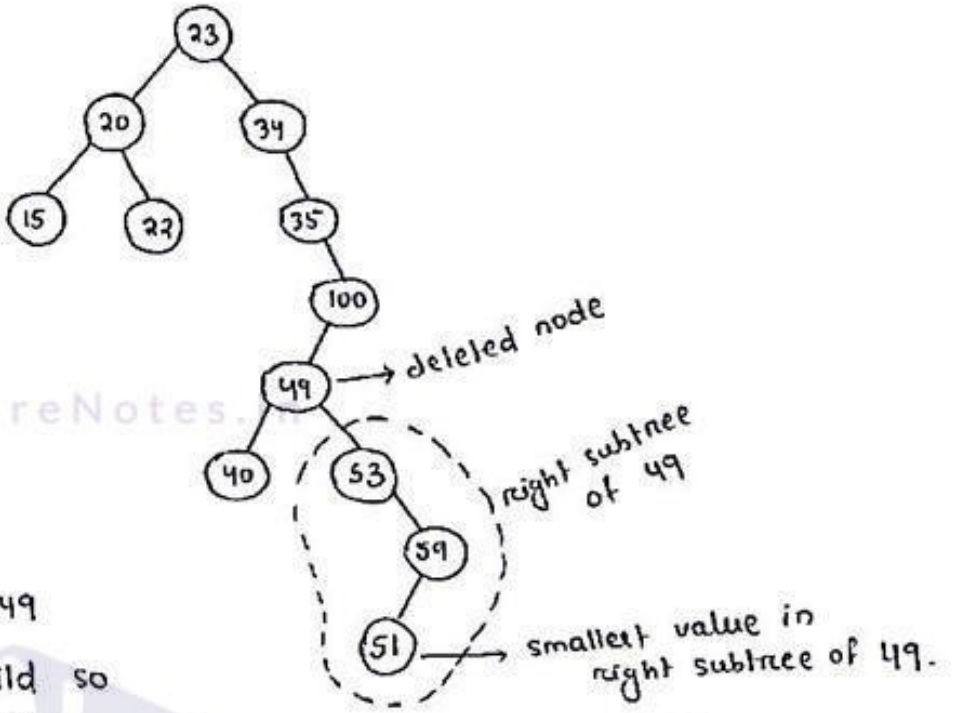
b) By the help of step1 and step2 the node having smallest value will be deleted.

c) replace N by the smallest node

d) delete (N)

Step4 : Exit.

Example :




deleted node = 49

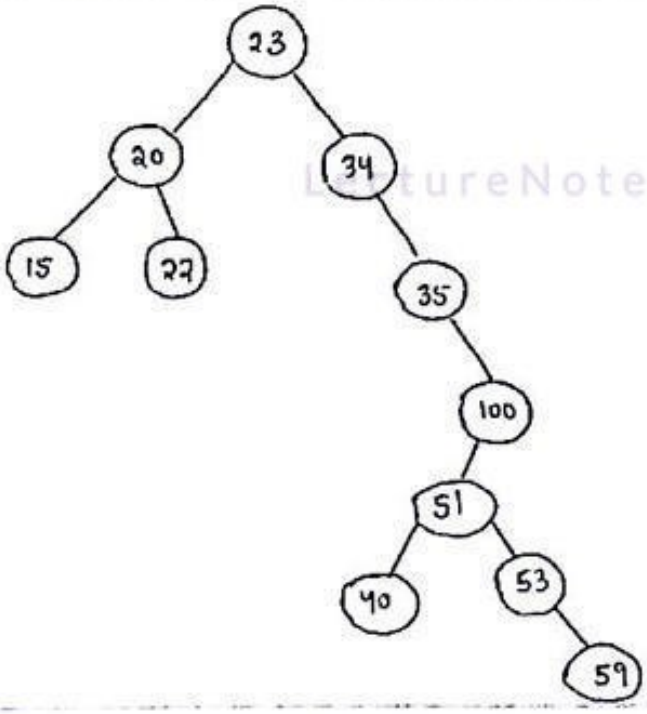
49 have 2 child so we go for step 3.

The right subtree of 49 is →


The smallest value in the right subtree of 49 = 51

So 51 is deleted from that position and 49 is replaced by 51

Now the tree becomes :



## Searching :

**Algorithm :**

PROCEDURE Search-in-BST ( t, item, left, right, info)

// t : Given Binary search tree's root node.

// item : searched item

// left : left child of a given node.

// right : right child of a given node.

// info : information part of each node.

Step 1 : if (t == null) then

i) print the tree is empty

ii) exit.

Step 2 : else

while (t → info != item && t != Null)

i) if ( item < t → info)

t = t → left.

ii) else

t = t → right

[ End of while ]

Step 3 : if (t → info = item)

i) print item is found having address t.

else

print item is not found.

Step 4 : exit.

**C - procedure :**

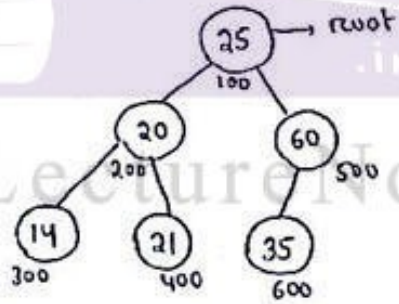
```
void search-BST ( struct node *t, int item)
{
// right, left, info are global variable. temp is also a global variable.
printf (" Enter the item to be searched");
scanf ("%d", &item);
tat
```

```
if ( t == NULL)
   { printf ("Tree is empty");
     exit();
   }
else { while ( t → info != item && t != NULL)
       {
     //  temp = t ;
         if ( item < t → info)
                t = t → left ;

           else t = t → right ;
         }
   if ( t → info == item)
       printf (" item is found in node having the address %u", t);

   else
       printf (" Item is not found");
   }
}
```

Example :




Search item = 14

t = 100.

while (25 != item && 100 != NULL)

temp = 100.

if (14 < 25)

t = 200

while ( 20 != item && 200 != null)

temp = 200

if ( 14 < 20)

t = 300

while ( 14 != 14 && 300 != 10)

(false)

∴ item is found in node having address 300.

## Advantages of BST :

→ Average time complexity for searching operation is $O(\log_2 n)$.

→ Insertion and deletion operation is quite easy.

## Disadvantage of BST :

→ If a binary search tree is right skewed or left skewed then its time complexity for searching operation in worst case is $O(n)$.
(∵ it is same as sequential search).